Intelligent Financial Insights for Firefly III

Real-Time Transaction Categorization & Monthly Spending Anomaly Detection

Complete Project Documentation
MLOps Capstone Project | Chameleon Cloud
Open Source Base: Firefly III (16k+ GitHub Stars)
April 2026

Table of Contents

1. Executive Summary

This project designs and deploys an end-to-end machine learning system into Firefly III, an open-source self-hosted personal finance manager with over 16,000 GitHub stars. The system integrates two complementary ML features—real-time transaction categorization and monthly spending anomaly detection—into Firefly III's existing workflow, hosted entirely on Chameleon Cloud infrastructure.

The project addresses a core pain point: users spend significant time manually categorizing transactions and often fail to notice unusual spending patterns until after the damage is done. By adding intelligent automation, Firefly III transforms from a passive transaction ledger into an active financial assistant.

The system is built on real-world spending data from the U.S. Bureau of Labor Statistics Consumer Expenditures Survey (CE Survey), with a full MLOps lifecycle including automated retraining, feedback loops, promotion gating, canary deployments, and drift monitoring.

1.1 Key Features

Feature	Model	Mode	Key Metric
Transaction categorization	DistilBERT (66M params)	Real-time (< 150ms)	Accuracy ≥ 90%, Macro-F1 ≥ 0.85
Spending anomaly detection	XGBoost + Isolation Forest	Batch (nightly)	Precision@5 ≥ 40%

1.2 Requirements Checklist

Requirement	How Satisfied
Hypothetical service with ML integration	Self-hosted Firefly III personal finance manager
Open source project ≥ 2.5k GitHub stars	Firefly III: 16,000+ stars
ML feature is complementary	Categorization + anomaly detection complement transaction management
Fully hosted on Chameleon	Single Chameleon VM with Docker
At least one trained/retrained model	DistilBERT (weekly) + XGBoost (monthly) retraining
Non-synthetic external dataset with known lineage	BLS Consumer Expenditures Survey (public domain, documented methodology)
Production feedback for retraining	User overrides, accepts, Helpful/Not Useful/Expected labels

2. Public-Facing Service

2.1 Service Overview

The public-facing service is a self-hosted personal finance management platform powered by Firefly III. Users access the platform through a web browser to track transactions, manage budgets, organize accounts, and view financial reports. Firefly III supports multi-currency tracking, recurring transactions, rule-based automation, and comprehensive reporting. The service runs on Chameleon Cloud in Docker containers with a PostgreSQL database backend.

2.2 Target Audience

The anticipated user base consists of 10–50 concurrent users:

- **Privacy-conscious individuals:** Users who prefer self-hosted solutions over commercial platforms like Mint or YNAB, wanting full control over their financial data.
- **Budget-focused households:** Individuals and small families actively managing budgets and seeking insight into spending behavior.
- **University students and young professionals:** Users establishing financial habits who benefit from automated spending analysis rather than manual review.
- **Small teams or co-living groups:** Shared households tracking communal expenses alongside personal budgets.

2.3 User Context and Pain Points

Users interact with Firefly III daily to weekly, logging transactions (manually or via bank imports), categorizing expenses, and reviewing dashboards. The ML features address two specific pain points:

1. Manual categorization is tedious and inconsistent. Users must assign categories to every transaction, and Firefly III supports custom categories per user—meaning every user has a different taxonomy. The DistilBERT model automates this by learning each user's personal category vocabulary.
2. Spending anomalies go unnoticed. Users often don't realize their spending in a category (e.g., dining, subscriptions) has crept upward until they do a manual end-of-month review. The XGBoost model provides early, automated detection of these patterns with explainable alerts.

3. External Dataset: Consumer Expenditures Survey

3.1 Dataset Overview

The primary external dataset is the Consumer Expenditures Survey (CE Survey), published by the U.S. Bureau of Labor Statistics (BLS). It is the most comprehensive federal survey of household spending in the United States, conducted continuously since 1980. The CE Survey reports detailed spending information from real people across hundreds of expenditure categories.

Aspect	Details
Publisher	U.S. Bureau of Labor Statistics (BLS)
Collection entity	U.S. Census Bureau (on behalf of BLS)
Collection method	Quarterly Interview Survey (~7,000 CUs) + Weekly Diary Survey (~7,000 CUs)
Population	~14,000 consumer units annually, representative of U.S. civilian non-institutional population
Time span used	2019–2024 (6 years, including post-pandemic recovery)
Granularity	Item-level expenditures coded with Universal Classification Codes (UCCs)
Number of UCC codes	~800 distinct expenditure categories in a hierarchical taxonomy
License	Public domain (U.S. government publication, no restrictions)
URL	https://www.bls.gov/cex/pumd_data.htm
Non-synthetic?	Yes — real household spending. Only merchant names are synthetically generated.

3.2 Why CE Survey Over Other Datasets

The CE Survey was selected based on course staff feedback identifying that Firefly III supports custom categories per user. A dataset with a fixed category set would contradict this design principle. The CE Survey's fine-grained UCC taxonomy with ~800 categories enables multi-label assignment where a single item can logically belong to multiple user-defined categories.

3.3 Files Used from the PUMD

The following files are downloaded from the BLS Public-Use Microdata (PUMD) page:

File	Full Name	Contents	Key Columns
MTBI	Monthly Tabulated Budget Item	Individual expenditures by UCC code per month	NEWID, UCC, COST, REFMO, REFYR
FMLI	Family-Level Interview	Household demographics, income, survey weights	NEWID, FINCBTXM, FAM_SIZE, AGE_REF, REGION

MEMI	Member-Level Interview	Individual member demographics	NEWID, MEMBNO, AGE, SEX, EDUCA
HG file	Hierarchical Groupings	UCC code → item name mapping (e.g., 020410 → Eggs)	Level, Title, UCC

For 2024 data, the Interview Survey zip contains quarterly files: fmli241x through fmli251, mtbi241x through mtbi251, and memi241x through memi251. All five quarters are combined for a full year. The hierarchical groupings file CE-HG-Integ-2024.txt (integrated) maps every UCC code in MTBI to its human-readable item name.

3.4 Data Pipeline: CE Survey to Training Data

3.4.1 Step 1: Persona Sampling

Each CE Survey respondent (consumer unit) represents a real spending persona. The pipeline randomly samples 500–1,000 personas from FMLI, weighted by survey weights (FINLWT21), extracting their demographics (income, family size, age, region, housing tenure) and complete expenditure records from MTBI.

3.4.2 Step 2: Item-to-Transaction Aggregation

CE Survey data is at the item level (e.g., “Eggs: $4.50”), but Firefly III transactions are at the merchant level (e.g., “KROGER #1247 $42.30”). The pipeline bridges this gap by grouping items from the same broad category within the same reporting period into single transactions, distributing quarterly totals across realistic purchase frequencies, and adding ±5–10% price noise.

3.4.3 Step 3: Multi-Label Category Assignment

Each generated transaction is assigned multiple possible categories using the UCC hierarchy (primary category from major group) and a pre-trained sentence-transformer model (secondary categories from semantic similarity). The training format becomes (transaction_description, [list_of_valid_categories]) enabling multi-label classification.

3.4.4 Step 4: Synthetic Merchant Name Generation

Merchant names are generated from a curated database of real merchant name patterns per UCC category, with regional variation (Midwest personas get Kroger/Meijer, West Coast gets Safeway/Trader Joes). Each persona is assigned 2–4 preferred merchants per category (70% of transactions), with 30% going to random alternatives. Names include realistic bank statement formatting (uppercase, store numbers, city/state abbreviations).

3.4.5 Step 5: Quality Validation

Generated data undergoes spending distribution validation (monthly totals match CE Survey averages), temporal consistency checks, merchant realism review (500-sample manual review), and multi-label coverage verification (≥2 valid categories per UCC).

3.5 Example Data Points

CE Survey Item (MTBI)	Amount	Generated Transaction	Possible Categories

UCC 020110: Cereal products	$82.40	KROGER #1247 SPRINGFIELD MO	Groceries, Food
UCC 350110: Electricity	$312.00	AMEREN MISSOURI ELEC	Utilities, Bills
UCC 320111: Soaps and detergents	$18.70	TARGET #0892 SPRINGFIELD MO	Household, Cleaning
UCC 580111: Health insurance	$1,450	BLUE CROSS BLUE SHIELD	Insurance, Healthcare
UCC 020410: Eggs	$4.50	TRADER JOES #482 CHICAGO IL	Groceries, Food, Eggs

4. Complementary ML Feature Design

4.1 Feature 1: Automated Transaction Categorization (Real-Time)

4.1.1 Problem Framing

Multi-label text classification: given a transaction description (and optional metadata), predict which user-defined categories it belongs to. The model outputs independent probabilities per category via sigmoid activation, suggesting multiple tags rather than forcing a single label.

4.1.2 Input

- **Primary:** Transaction description string (e.g., “AMZN MKTP US*2K4MR1LO3”). Preprocessed: lowercased, special characters normalized, reference numbers replaced with [NUM] token, WordPiece tokenized to max_length=64.
- **Auxiliary:** Log-normalized amount (1-dim), one-hot currency (20-dim), one-hot country (50-dim). Concatenated with [CLS] embedding before classification head.

4.1.3 Output

- **Category probabilities:** Independent sigmoid outputs for each category. Categories with probability > 0.5 are suggested.
- **Confidence score:** Maximum sigmoid probability. Below 0.7 triggers abstention — user is prompted to manually categorize.
- **Top-3 suggestions:** Displayed as clickable tags for user selection.

4.1.4 Feedback Mechanism

- **User overrides (strong labels, weight=1.0):** User changes the predicted category to a different one.
- **Accepted predictions (weak labels, weight=0.3):** User does not change the prediction.

All feedback is stored with the full feature context for retraining.

4.2 Feature 2: Monthly Spending Trend Detection (Batch)

4.2.1 Problem Framing

Regression + anomaly detection: predict expected spending per user-category-month, flag deviations exceeding a configurable threshold. The model answers: “Is this user’s current spending in this category significantly different from their historical baseline?”

4.2.2 Input: 20-Feature Vector

#	Feature	Type	Description
1	current_spend	float	Total spend in category this month
2	rolling_mean_1m	float	Mean spend, preceding 1 month
3	rolling_mean_3m	float	Mean spend, preceding 3 months
4	rolling_mean_6m	float	Mean spend, preceding 6 months

5	rolling_std_3m	float	Std deviation, preceding 3 months
6	rolling_std_6m	float	Std deviation, preceding 6 months
7	deviation_ratio	float	current_spend / rolling_mean_3m (capped at 10)
8	share_of_wallet	float	Category spend / total monthly spend
9	hist_share_of_wallet	float	Historical avg share-of-wallet (6 months)
10	txn_count	int	Transaction count this month
11	hist_txn_count_mean	float	Mean transaction count (3 months)
12	avg_txn_size	float	current_spend / txn_count
13	hist_avg_txn_size	float	Historical mean transaction size
14	days_since_last_txn	int	Days since last transaction in category
15	month_of_year	int	Calendar month (1–12, seasonality)
16	spending_velocity	float	Cumulative spend at day 15 / rolling_mean_3m
17	weekend_txn_ratio	float	Weekend txns / total txns
18	total_monthly_spend	float	Total spend across all categories
19	elevated_cat_count	int	Categories above 1.2x their mean
20	budget_utilization	float	Spend / budget (0 if no budget set)

4.2.3 Output

- **anomaly_score:** Combined score from XGBoost residual (0.6 weight) + Isolation Forest (0.4 weight), normalized to [0,1].
- **predicted_spend:** XGBoost's predicted expected spending amount.
- **direction:** +1 (overspend) or –1 (underspend).
- **magnitude_pct:** Percentage deviation from predicted value.
- **top_factors:** Top 3 features by SHAP value, explaining the anomaly.

4.2.4 Outcome Translation

Raw scores are translated into user-facing alerts through thresholding (score > 0.6), severity bucketing (Low/Medium/High), natural language summaries, and a "Spending Insights" dashboard widget. Users provide feedback via Helpful / Not Useful / Expected buttons.

5. Training Role: Model Architectures & Training

5.1 Feature 1: DistilBERT Categorization

5.1.1 Architecture

- **Base model:** distilbert-base-uncased (66M parameters, 6 transformer layers, 12 attention heads, hidden dimension 768).
- **Classification head:** [CLS] embedding (768) + auxiliary features (71) = 839-dim → Linear(839, 256) → ReLU → Dropout(0.3) → Linear(256, N_categories) → Sigmoid.
- **Output activation:** Sigmoid (multi-label) instead of softmax (single-label), matching Firefly III's custom user-defined categories.

5.1.2 Training Procedure

Phase 1 — Pre-training on CE Survey generated data:

Hyperparameter	Value
Optimizer	AdamW (β1=0.9, β2=0.999, weight_decay=0.01)
Learning rate	2e-5 with linear warmup (10% of steps) then linear decay
Batch size	32
Epochs	3–5 with early stopping (patience=1)
Max sequence length	64 tokens
Loss function	BCEWithLogitsLoss (binary cross-entropy for multi-label)
Mixed precision	FP16 via PyTorch AMP
Training time	~4–6 hours on Chameleon GPU (T4/V100)

Phase 2 — Fine-tuning with production feedback:

Hyperparameter	Value
Learning rate	5e-6 (10x lower to prevent catastrophic forgetting)
Data mixing	50% production + 50% external (preserves general patterns)
Layer freezing	Layers 0–3 frozen; layers 4–5 + head trainable
Sample weighting	User overrides: 1.0, Accepted: 0.3, External: 0.5

5.1.3 Evaluation & Promotion Gating

- Accuracy ≥ 90% on test set
- Macro-F1 ≥ 0.85
- No individual category recall below 0.70
- Abstention rate between 5% and 20%
- Inference latency p95 < 150ms

- Manual spot-check of 50 random predictions

5.2 Feature 2: XGBoost Trend Detection

5.2.1 Architecture

- **Primary model:** XGBoost gradient-boosted regression (reg:squarederror). 100–300 trees, max_depth=6, learning_rate=0.05 (global) / 0.01 (per-user), L1=0.1, L2=1.0, subsample=0.8, colsample_bytree=0.8.
- **Ensemble member:** Isolation Forest (100 trees, contamination=0.1). Final score = 0.6 × XGBoost_residual + 0.4 × IsolationForest_score.

5.2.2 Training Procedure

Phase 1 — Global pre-training on CE Survey data (~180,000 examples, chronological split). Phase 2 — Per-user fine-tuning once user has 3+ months of history (50–100 additional trees, learning_rate=0.01, increased regularization). Training time: ~15 minutes global, < 5 seconds per user.

5.2.3 Evaluation & Promotion Gating

- RMSE ≤ current production model (or within 5% tolerance)
- MAE does not increase by > 10%
- Precision@5 ≥ 40% on historical flagged anomalies
- No single category RMSE > 3x global average
- Feature importance rankings stable (no unexplained shifts)

6. Serving Role: Operational Requirements

6.1 Operational Specifications

Metric	Specification
Categorization inference mode	Real-time: REST API triggered on each new transaction
Categorization latency target	< 150ms at p95
Trend detection mode	Batch: nightly scheduled run at 2:00 AM
Batch throughput	~50 users × ~15 categories = 750 predictions in < 5 minutes
On-demand refresh	User-triggered: < 3 seconds for all categories for one user
Peak requests/second	2–5 req/s (multiple users refreshing simultaneously)
Availability target	99% uptime
Model loading time	< 2 seconds at service start

6.2 Model Output to Outcome Translation

Categorization: The top suggested categories appear as clickable tags when a user creates a transaction. If confidence < 0.7, the user is prompted to select manually (abstention). High-confidence predictions auto-fill the category field.

Trend detection: Alerts appear in a “Spending Insights” dashboard widget with severity tiers (Low: informational badge, Medium: highlighted card, High: banner + optional email), sparkline charts showing 6-month category history, and action buttons for feedback.

7. Data Role: Data Flow & Leakage Prevention

7.1 Data Flow

7.1.1 Data Arrival

3. User-entered transactions: Manually logged through Firefly III web interface with date, amount, category, accounts, tags. Written immediately to PostgreSQL.
4. Automated imports: CSV imports and API-based ingestion. Transactions pass through Firefly III's rule engine for auto-categorization before database commit.

7.1.2 Inference Pipeline

5. For categorization (real-time): New transaction triggers API call → text preprocessed → DistilBERT inference → category suggestions returned → displayed to user.
6. For trend detection (batch): Nightly cron extracts transactions for all users (current month + 6 months history) → 20-feature vectors computed → XGBoost + Isolation Forest inference → anomaly alerts written to alerts table → dashboard queries alerts table.

7.1.3 Training Data Pipeline

7. Historical transactions extracted and aggregated monthly per category.
8. Same 20-feature computation applied to each historical month.
9. Feedback labels merged (Expected → adjust target, Helpful → confirm).
10. Processed dataset serialized to Parquet for efficient training.

7.2 Candidate Selection

- **Minimum history:** Only user-category pairs with ≥3 months of history included.
- **Active categories:** Categories with < 2 transactions/month excluded.
- **Outlier review:** Spending > 10x category median flagged for manual review.
- **Temporal completeness:** Partial months (user joined mid-month) excluded.

7.3 Leakage Prevention

- **Strict temporal splitting:** Train/validation/test split chronologically, never randomly.
- **Rolling window features only:** All features computed using data up to (not including) prediction month.
- **Feedback label timing:** Feedback only used in retraining after the prediction date.
- **Cross-user isolation:** Per-user models use only that user's data (except global pre-training).
- **Shared feature pipeline:** Identical feature computation for training and serving (no training-serving skew).
- **No target leakage:** Next month's actual spending never in feature set. Budget utilization uses start-of-month budget.

8. DevOps/Platform Role: Infrastructure & Scaling

8.1 Model Freshness Requirements

Component	Frequency	Trigger	Justification
DistilBERT categorization	Weekly	Scheduled Sundays 3 AM	Feedback accumulates daily; weekly captures patterns
DistilBERT categorization	Ad-hoc	Override rate > 15%	Spike indicates model degradation
XGBoost trend detection	Monthly	1st of month, 3 AM	Aligns with monthly spending cycles
XGBoost trend detection	Ad-hoc	10+ feedback signals	Enough new labeled data to update
Isolation Forest	Monthly	Alongside XGBoost	Must stay synchronized with ensemble
Global XGBoost baseline	Quarterly	Every 3 months	Cross-user patterns evolve slowly

8.2 Retraining Automation Pipeline

11. Trigger detection (cron or threshold monitor).
12. Data snapshot: Versioned copy of training data + feedback.
13. Feature computation: Updated feature vectors using shared pipeline.
14. Model training: Phase 2 fine-tuning with defined hyperparameters.
15. Evaluation: Metrics checked against promotion gates.
16. Shadow deployment: New model runs 24 hours in shadow mode (predictions logged, not shown).
17. Canary rollout: 10% of traffic for 48 hours. If override/feedback rates spike > 5pp, auto-halt.
18. Promotion: Model registered with full metadata (artifact, data hash, hyperparameters, metrics).

8.3 Data Drift Detection

- **PSI (Population Stability Index):** Computed weekly per input feature. PSI > 0.2 triggers alert; > 0.25 triggers immediate retraining.
- **Concept drift:** Running accuracy/RMSE on recent data tracked. Sustained decline over 2+ weeks triggers investigation.
- **Label drift:** Distribution of user-selected categories monitored for significant shifts.

8.4 Scaling Requirements

Scenario	Characteristics	Resources

Typical usage	Nightly batch: 50 users, 750 predictions. 2–3 on-demand/hour	1 vCPU, 2 GB RAM
Peak usage	End of month: all 50 users + nightly batch	2 vCPU, 4 GB RAM
Retraining	Monthly: 50 per-user models, ~10 min total	4 vCPU, 8 GB RAM
Global training	Quarterly: full dataset, ~30 min	4 vCPU, 8 GB RAM

8.5 Right-Sizing Strategy

- **Single-node deployment:** All components on one Chameleon VM (4 vCPU, 8 GB RAM, 80 GB SSD).
- **Time-based isolation:** Training at 3 AM, batch inference at 2 AM, on-demand during business hours.
- **Vertical scaling path:** Upgrade to 8 vCPU, 16 GB RAM before adding horizontal complexity. System handles ~500 users without distributed computing.
- **Model storage:** ~50–200 KB per XGBoost model, ~100–300 KB per Isolation Forest. Total < 25 MB for 50 users.

8.6 Monitoring

- Batch job completion time and success/failure status.
- Inference latency percentiles (p50, p95, p99).
- Training pipeline duration and evaluation metrics per cycle.
- Disk, CPU, memory utilization on Chameleon VM.
- Data freshness: time since last feature cache update per user.

9. Model Registry & Reproducibility

Every trained model is logged with complete metadata for reproducibility and audit:

Field	Type	Description
model_id	UUID	Unique identifier for this model version
model_type	enum	DISTILBERT_CATEGORIZATION / XGBOOST_TREND / ISOLATION_FOREST
version	semver	Incremented on each retrain (e.g., 2.3.1)
user_id	UUID/NULL	NULL for global models; user UUID for per-user models
training_data_hash	SHA-256	Hash of training dataset for exact reproducibility
feature_version	string	Version of feature engineering pipeline
hyperparameters	JSON	Complete hyperparameter configuration
eval_metrics	JSON	All evaluation metrics
training_timestamp	datetime	When training completed
artifact_path	string	Path to serialized model file
status	enum	TRAINING / SHADOW / CANARY / PRODUCTION / RETIRED
predecessor_id	UUID	Model version this replaced

Any model version can be exactly reproduced by loading the dataset identified by training_data_hash, applying the feature pipeline identified by feature_version, using the stored hyperparameters, and training with the same random seed (42).

10. Project Summary for Resume

10.1 Bullet-Point Version

Intelligent Financial Insights for Firefly III — MLOps Capstone Project

DistilBERT | XGBoost | Chameleon Cloud | Python | Docker | PyTorch

- Architected a production ML system integrating two complementary models into Firefly III (16k+ stars), a self-hosted personal finance manager, deployed on Chameleon Cloud.
- Built real-time transaction categorization using fine-tuned DistilBERT with multi-label classification, achieving >90% accuracy with confidence-based abstention.
- Developed batch spending anomaly detection using XGBoost + Isolation Forest ensemble on a 20-feature vector, surfacing explainable insights via SHAP values.
- Designed a data pipeline transforming BLS Consumer Expenditures Survey (real household spending) into synthetic merchant-level transactions with multi-label categories.
- Implemented full MLOps lifecycle: automated weekly/monthly retraining, promotion gating, shadow deployment, canary rollout, drift monitoring (PSI), model registry with versioning, and instant rollback.

10.2 One-Liner

Built a production MLOps pipeline deploying DistilBERT + XGBoost into an open-source finance app (Firefly III), with automated retraining, feedback loops, drift monitoring, and canary deployments on Chameleon Cloud.

10.3 Skills & Technologies

Category	Technologies
ML Models	DistilBERT, XGBoost, Isolation Forest, Sentence-Transformers
ML Concepts	Transfer learning, fine-tuning, multi-label classification, anomaly detection, SHAP explainability
MLOps	Automated retraining, model registry, promotion gating, canary deployments, drift detection (PSI), rollback
Data Engineering	ETL (CE Survey → synthetic transactions), feature stores, temporal splitting, leakage prevention
Infrastructure	Chameleon Cloud, Docker, PostgreSQL, cron orchestration, REST API
Languages	Python, PyTorch, scikit-learn, pandas, Hugging Face Transformers, XGBoost